

Quality Assurance & Release Validation Module (Manual + Functional + Performance)

Owned by All Interns (Shared Responsibility)

No Automation Required

1. Executive Summary

The **Quality Assurance & Release Validation Module** ensures that all RhoFi features developed by interns—Usage Analytics, Finance 360, Notifications, Blog Engine—meet enterprise quality standards through:

- Manual functional testing
- UI/UX validation
- API correctness checks
- Basic performance testing
- Release readiness evaluations
- Weekly reporting

This is foundational for **stability, reliability, and user trust**, especially as multiple independent modules operate in parallel.

2. Business Problem

Without structured QA:

- Features may break unexpectedly
- UI may be inconsistent
- APIs may behave incorrectly
- Performance may degrade
- Intern modules may conflict with each other

Manual QA ensures the product is **release-ready and stable**, even without automation capabilities.

3. Module Objectives

- ✓ Validate all features manually before merge
 - ✓ Ensure UI/UX consistency across modules
 - ✓ Check business logic correctness
 - ✓ Verify API inputs/outputs
 - ✓ Perform basic performance testing
 - ✓ Create QA checklists for every release
 - ✓ Prevent regressions through structured test cycles
-

4. User Personas

Persona	Role	Needs
Intern QA Team	Owners	Clear structured testing plan
Frontend Developers	Receivers	Feedback on UI/UX issues
Backend Team (Shafvan)	Receivers	API correctness checks
Business Team (Sekar & Rajani)	Validation	Correctness of insights & logic
Users	End consumers	Stable, consistent experience

5. Functional Requirements (Manual Testing)

5.1 Manual Functional Testing

Interns must manually test:

A. Usage Analytics Dashboard

- Event logging
- Filters
- Dashboard UI accuracy
- Data consistency
- Export functionality

B. Finance 360 Insights

- Income & expense input
- Ratio calculations
- Net worth tracker
- Insights interpretation
- Charts rendering properly
- Category spend logic

C. Notifications Module

- Trigger events manually
- Validate in-app notifications
- Read/unread behavior
- Settings toggles

D. Automated Research & Blog Engine

- Draft generation flow
- Draft editing
- Publishing
- Blog rendering
- Scheduling

Each test must capture:

- Expected output
- Actual output
- Screenshots
- Severity
- Reproduction steps

5.2 UI/UX Validation

Interns must verify:

- Consistency with design guidelines (Gopal)
- Component alignment
- Mobile responsiveness
- Font, color, spacing compliance
- Proper error messages

5.3 API Testing (Manual)

Using Postman or Swagger:

Validate:

- Correct status codes
- Input validation errors
- Response structure

- Business rules
- Edge-case behavior

Example API Tests:

- Missing fields
 - Invalid data types
 - Large payloads
 - SQL injection safety (basic negative testing)
-

5.4 Basic Performance Testing

Scope:

- Page load time (should load < 2 seconds)
- API latency (< 300ms for GETs)
- Light concurrent access (~10 parallel requests)

Tools (optional, recommended):

- Postman collection runner
- Browser DevTools
- Simple JMeter script (optional)

Performance Checks:

- Chart rendering speed
 - Notification delivery latency
 - Finance insight calculations under load
-

5.5 Cross-Module Integration Testing

Example flows:

1. **Blog Published** → **Notification Trigger** → **User sees notification**
2. **Usage Event Logged** → **Appears on Analytics Dashboard**
3. **Finance Data Update** → **Insights Refresh**

Each flow must be validated manually.

6. Non-Functional Requirements

Category	Requirement
Accuracy	100% validation of business rules
Consistency	Test all modules weekly
Usability	UI must be smooth, no broken flows
Performance	Page load < 2s, APIs < 300ms
Stability	No crashes for normal usage

7. Test Artifacts

Interns must create the following:

Artifact	Description
Test Plan	Scope, strategy, timelines
Test Cases	Manual steps, expected results
Bug Report	Clear & structured issue logging
Release Checklist	Items to verify before launch
Weekly QA Report	Summary of tests done, bugs found, status

8. Test Case Structure (Required Format)

Test Case ID:

Module:

Step-by-Step Actions:

Expected Result:
Actual Result:
Severity:
Status:
Screenshot (if applicable):
Notes:

9. Release Readiness Checklist

Must Pass:

- All critical test cases
- UI responsive checks
- All APIs respond correctly
- Business logic validated
- Notifications tested
- No critical or high-severity open bugs
- Performance thresholds met

Release can **proceed only** if:

- Foundational flows are stable
- UI is visually consistent
- No blocking issues remain

10. Weekly Reporting Structure

Every Friday, interns must submit a **QA Summary Report**:

Report Must Include:

- Modules tested
- Number of test cases executed
- Test cases passed/failed

- New bugs found
- Bugs fixed
- Key risks
- Next week plan

Reviewed by:

- **Sekar** (Business validation)
- **Boopalan** (Infra dependencies)
- **Shafvan** (Backend issues)
- **Gopal** (UI issues)

11. Dependencies

Area	Contact
Business validation	Sekar / Rajani
API behavior	Shafvan
UI consistency	Gopal
Module explanations	Boopalan / Shafvan / Gopal

12. Risks & Mitigation

Risk	Mitigation
Testing delays	Clearly divide testing among interns
Missing scenarios	Weekly review with Sekar
UI defects	Early review cycles with Gopal
Performance bottlenecks	Identify early and escalate

13. Acceptance Criteria

- ✓ Complete test coverage for all intern modules
- ✓ All critical bugs resolved before merge

- ✓ Weekly QA reports submitted
 - ✓ Performance benchmarks met
 - ✓ Release checklist completed
 - ✓ Interns understand end-to-end product flows
-

14. 1-Month Delivery Timeline

Week 1

- Test plan creation
- Module understanding
- Create test cases

Week 2

- Full functional testing of each module
- UI validation
- API validation

Week 3

- Performance testing
- Integration flows
- Regression round

Week 4

- Final QA cycles
- Release readiness
- PDF report & Demo